1. Fraud Transaction Detection (Linear Search)

Problem Statement

A bank stores transaction IDs in the order they occur. Since the transactions are not sorted, the

The security team needs to check if a suspicious transaction ID exists.

Write a program using Linear Search.

2. Server Log Search (Binary Search)

Problem Statement

A cloud company stores server log IDs in sorted order. Engineers need to quickly verify if a log ID

exists.

Write a program using Binary Search.

3. Sort Employee Salaries (Bubble Sort)

Problem Statement

A company wants to sort employee salaries in ascending order for payroll analysis.

Write a program using Bubble Sort.

4. Sort Website Response Time (Insertion Sort)

Problem Statement

A web monitoring system records website response times. New data comes continuously and must

be inserted into the correct position.

Write a program using Insertion Sort.

5. Prioritize Bug Reports (Selection Sort)

Problem Statement

A software company assigns priority scores to bug reports. To resolve bugs efficiently, they want to

sort priorities in ascending order.

Write a program using Selection Sort.

Problem 1 - Linear search

- Topic Description / Definition.

  Linear search is a straightforward searching algorithms that checks every single elements in a list sequentially, from the very beginning to the very end, untill a match is found or the list concludes. It is highely effective for unsorted data.

- Logic and step by step Explanation.

1- Read the total no. of transactions (N) and the array of transaction IDs.
2- Read the target suspicious transaction ID to look for.
3- Initialized a boolean flag Found = false.
4- Run a Loop through the list. If any transaction matches the target ID, set Found = True and terminate the Loop (break).
5- Check flag, if true print "Fraud Transaction found"; Otherwise, print "Transaction Not found".

week 2 module

Update

EXPLORER

WEEK 2 MODULE

problem1.py

Welcome

problem1.py

problem1.py > ...

```
1  N = int(input())
2  transactions = list(map(int, input().split()))
3  suspicious_id = int(input())
4
5  found = False
6  for tx in transactions:
7      if tx == suspicious_id:
8          found = True
9          break
10
11 print("Fraud Transaction Found" if found else "Transacti
```

CHAT

SESSIONS

PROBLEMS OUTPUT TERMINAL ··· Python

```
/usr/local/bin/python3 "/Users/macbook/Desktop/week 2 module/problem1.py"

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
MacBooks-MacBook-Air:week 2 module macbook$ /usr/local/bin/python3 "/Users
/macbook/Desktop/week 2 module/problem1.py"
5
1201 1305 1450 1408 1502
1450
Fraud Transaction Found
MacBooks-MacBook-Air:week 2 module macbook$
```

OUTLINE

TIMELINE

Tip: Try the Plan agent to research and plan before implementing changes.

problem1.py

Describe what to build

Auto

Local | Default Approvals

0 0 Ln 11, Col 71 Spaces: 4 UTF-8 Python Python 3.14.6

Problem 2 - Binary Search.

- Topic Description Defination

Binary search is a highly efficient search algorithm used exclusively on sorted arrays. It works by repeatedly dividing the search interval in half. If the target value is less the middle element, the narrow search space shifts to lower half; otherwise, it shifts to the upper half

- Logic and step by step Explanation.

1- Establish two pointer: Low at index 0 and high at index N-1

2- Loop while Low <= high. find the middle element using mid = (Low + high) / 2

3- If the element at mid equal the target, set found = True and break.

4- If the element at mid is small than the target, discard the left half by updating Low = mid + 1

5- If the element at mid is large, discard the right half by updating high = mid - 1.

EXPLORER

WEEK 2 MODULE
- problem1.py
- problem2.py

Welcome | problem1.py | problem2.py

problem2.py > ...

```python
def binary_search():
    # Step 1: Read inputs
    N = int(input())
    logs = list(map(int, input().split()))
    target = int(input())

    low = 0
    high = N - 1
    found = False

    while low <= high:
        mid = (low + high) // 2
        if logs[mid] == target:
            found = True
            break
        elif logs[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    if found:
        print("Log Found")
    else:
        print("Log Not Found")

if __name__ == "__main__":
    binary_search()
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

Python

```
/usr/local/bin/python3 "/Users/macbook/Desktop/week 2 module/problem2.py"

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
MacBooks-MacBook-Air:week 2 module macbook$ /usr/local/bin/python3 "/Users/macbook/Desktop/week 2 module/problem2.py"
7
1001 1005 1010 1015 1020 1030 1040
1015
Log Found
MacBooks-MacBook-Air:week 2 module macbook$
```

CHAT

SESSIONS

Tip: Try the Plan agent to research and plan before implementing changes.

problem2.py

Describe what to build

Auto

Local | Default Approvals

OUTLINE

TIMELINE

Problem 3 - Bubble sort.

- Topic Description / Definition -

Bubble sort is a simple comparison-based sorting algorithm. It repeatedly steps through the list, compare adjacent element, and swaps them if they are in the wrong order. This process is repeated untill the entire list is sorted, with the largest values "bubbling" to the end in each pass.

- Logic and step by step Explanation -

1- Take the total count of employees salaries (N) and the array of salaries as input.
2- Run an Outer loop from 0 to N-1 track the no. of passes.
3- Run an inner nested loop to compare adjacent items up to the unsorted boundary (N-i-1)
4- if salaries[j] > salaries[j+1], swap their position.
5- print the completely sorted array.

week 2 module

Update

EXPLORER

WEEK 2 MODULE

- problem1.py
- problem2.py
- problem3.py

Welcome | problem1.py | problem2.py | problem3.py

problem3.py > ...

```python
N = int(input())
salaries = list(map(int, input().split()))

for i in range(N):
    for j in range(0, N - i - 1):
        if salaries[j] > salaries[j + 1]:
            salaries[j], salaries[j + 1] = salaries[j + 1], salaries[j]

print(*(salaries))
```

CHAT

SESSIONS

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python

```
/usr/local/bin/python3 "/Users/macbook/Desktop/week 2 module/problem3.py"

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
MacBooks-MacBook-Air:week 2 module macbook$ /usr/local/bin/python3 "/Users/macbook/Desktop/week 2 module/problem3.py"
5
45000 32000 55000 28000 60000
28000 32000 45000 55000 60000
MacBooks-MacBook-Air:week 2 module macbook$
```

Tip: Try the Plan agent to research and plan before implementing changes.

problem3.py

Describe what to build

Auto

Local | Default Approvals

OUTLINE

TIMELINE

0 0

Ln 6, Col 42 Spaces: 4 UTF-8 LF { } Python Python 3.14.6

Problem 4 - Insertion sort

- Topic Description / Definition -

Insertion sort builds a sorted array item by item it picks one element at a time from the unsorted segment and inserted it into exact relative position with in the already sorted portion of the array, shifting larger elements forward as necessary.

- Logic and step by step Explanation -

1- Start the loop from the second element (index = 1) up to N-1

2- Store the current element in a variable called key.

3- Compare key with element in the sorted subarray to its left (from index i-1 down to 0)

4- while the left element is greater than key, shift that element one position to the right.

5- Drop the key value into ils correct vacant index slot.

week 2 module

Update

EXPLORER

WEEK 2 MODULE

- problem1.py
- problem2.py
- problem3.py
- problem4.py

Welcome | problem1.py | problem2.py | problem3.py | problem4.py

problem4.py > ...

```python
N = int(input())
times = list(map(int, input().split()))

for i in range(1, N):
    key = times[i]
    j = i - 1
    while j >= 0 and times[j] > key:
        times[j + 1] = times[j]
        j -= 1
    times[j + 1] = key

print(*(times))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python

```
/usr/local/bin/python3 "/Users/macbook/Desktop/week 2 module/problem4.py"

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
MacBooks-MacBook-Air:week 2 module macbook$ /usr/local/bin/python3 "/Users/macbook/Desktop/week 2 module/problem4.py"
5
250 120 320 180 100
100 120 180 250 320
MacBooks-MacBook-Air:week 2 module macbook$
```

OUTLINE

TIMELINE

CHAT

SESSIONS

Tip: Try the Plan agent to research and plan before implementing changes.

problem4.py

Describe what to build

Auto

Local | Default Approvals

0 0

Ln 12, Col 16 Spaces: 4 UTF-8 LF { } Python Python 3.14.6

Problem 5 - Selection Sort.

- Topic Description / Definition.

Selection sort sorts an array by repeatedly finding the absolute minimum element from the unsorted section and swapping it with the first element of that unsorted section. This expands the sorted boundary by one element per iteration.

- Logic and step by step Explanation.

1- Maintain an outer loop tracking the current index position (i) being filled

2- Assume the current position i contain the minimum value (min_idx = i).

3- Run an inner loop from i + 1 to N-1 to check if any smaller priority value exists.

4- if a small value is detected, update min_idx to that index

5- swap the minimum value found with the element at position i.

week 2 module

Update

EXPLORER

WEEK 2 MODULE

- problem1.py
- problem2.py
- problem3.py
- problem4.py
- problem5.py

problem1.py | problem2.py | problem3.py | problem4.py | problem5.py

problem5.py > ...

```python
N = int(input())
raw_input = input().strip()

if ' ' in raw_input:
    priorities = list(map(int, raw_input.split()))
else:
    priorities = [int(char) for char in raw_input]

for i in range(N):
    min_idx = i
    for j in range(i + 1, N):
        if priorities[j] < priorities[min_idx]:
            min_idx = j
    priorities[i], priorities[min_idx] = priorities[min_idx], priorities[i]

if ' ' in raw_input:
    print(*(priorities))
else:
    print("".join(map(str, priorities)))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Python

```
/usr/local/bin/python3 "/Users/macbook/Desktop/week 2 module/problem5.py"

The default interactive shell is now zsh.
To update your account to use zsh, please run `chsh -s /bin/zsh`.
For more details, please visit https://support.apple.com/kb/HT208050.
MacBooks-MacBook-Air:week 2 module macbook$ /usr/local/bin/python3 "/Users/macbook/Desktop/week 2 module/problem5.py"
5
92815
12589
MacBooks-MacBook-Air:week 2 module macbook$
```

CHAT

SESSIONS

Tip: Try the Plan agent to research and plan before implementing changes.

problem5.py

Describe what to build

Auto

Local | Default Approvals

OUTLINE

TIMELINE

Ln 19, Col 41 Spaces: 4 UTF-8 LF { } Python Python 3.14.6